

Agents

Agents are the core building block in your apps. An agent is a large language model (LLM) configured with instructions, tools, and optional runtime behavior such as handoffs, guardrails, and structured outputs.

Use this page when you want to define or customize a single base `Agent` rather than a `SandboxAgent`. If you are deciding how multiple agents should collaborate, read [Agent orchestration](#). If the agent should run inside an isolated workspace with manifest-defined files and sandbox-native capabilities, read [Sandbox agent concepts](#).

The SDK uses the Responses API by default for OpenAI models, but the distinction here is orchestration: `Agent` plus `Runner` lets the SDK manage turns, tools, guardrails, handoffs, and sessions for you. If you want to own that loop yourself, use the Responses API directly instead.

Choose the next guide

Use this page as the hub for agent definition. Jump to the adjacent guide that matches the next decision you need to make.

If you want to...	Read next
Choose a model or provider setup	Models
Add capabilities to the agent	Tools
Run an agent against a real repo, document bundle, or isolated workspace	Sandbox agents quickstart
Decide between manager-style orchestration and handoffs	Agent orchestration

If you want to...	Read next
Configure handoff behavior	Handoffs
Run turns, stream events, or manage conversation state	Running agents
Inspect final output, run items, or resumable state	Results
Share local dependencies and runtime state	Context management

Basic configuration

The most common properties of an agent are:

Property	Required	Description
<code>name</code>	yes	Human-readable agent name.
<code>instructions</code>	no	System prompt or dynamic instructions callback. Strongly recommended. See Dynamic instructions .
<code>prompt</code>	no	OpenAI Responses API prompt configuration. Accepts a static prompt object or a function. See Prompt templates .
<code>handoff_description</code>	no	Short description exposed when this agent is offered as a handoff target.
<code>handoffs</code>	no	Delegate the conversation to specialist agents. See handoffs .
<code>model</code>	no	Which LLM to use. See Models .
<code>model_settings</code>	no	Model tuning parameters such as <code>temperature</code> , <code>top_p</code> , and <code>tool_choice</code> .

Property	Required	Description
<code>tools</code>	no	Tools the agent can call. See Tools .
<code>mcp_servers</code>	no	MCP servers that provide MCP-backed tools to the agent. See the MCP guide .
<code>mcp_config</code>	no	Fine-tune how MCP tools are prepared, such as converting their schemas to strict mode and formatting MCP failures. See the MCP guide .
<code>input_guardrails</code>	no	Guardrails that run on the first user input for this agent chain. See Guardrails .
<code>output_guardrails</code>	no	Guardrails that run on the final output for this agent. See Guardrails .
<code>output_type</code>	no	Structured output type instead of plain text. See Output types .
<code>hooks</code>	no	Agent-scoped lifecycle callbacks. See Lifecycle events (hooks) .
<code>tool_use_behavior</code>	no	Control whether tool results loop back to the model or end the run. See Tool use behavior .
<code>reset_tool_choice</code>	no	Reset <code>tool_choice</code> after a tool call (default: <code>True</code>) to avoid tool-use loops. See Forcing tool use .

```
from agents import Agent
from agents.decorators import tool

@tool
def get_weather(city: str) -> str:
    """returns weather info for the specified city."""
    return f"The weather in {city} is sunny"

agent = Agent(
```

```
name="Haiku agent",
instructions="Always respond in haiku form",
model="gpt-5-nano",
tools=[get_weather],
)
```

Everything in this section applies to `Agent`. `SandboxAgent` builds on the same ideas, then adds `default_manifest`, `base_instructions`, `capabilities`, and `run_as` for workspace-scoped runs. See [Sandbox agent concepts](#).

Prompt templates

You can reference a prompt template created in the OpenAI platform by setting `prompt`. This works when OpenAI models are accessed through the Responses API.

To use it, please:

1. Go to <https://platform.openai.com/playground/prompts>
2. Create a new prompt variable, `poem_style`.
3. Create a system prompt with the content:

```
Write a poem in {{poem_style}}
```

4. Run the example with the `--prompt-id` flag.

```
from agents import Agent

agent = Agent(
    name="Prompted assistant",
    prompt={
        "id": "pmpt_123",
        "version": "1",
        "variables": {"poem_style": "haiku"},
    },
)
```

You can also generate the prompt dynamically at run time:

```
from dataclasses import dataclass

from agents import Agent, GenerateDynamicPromptData, Runner

@dataclass
class PromptContext:
    prompt_id: str
    poem_style: str

async def build_prompt(data: GenerateDynamicPromptData):
    ctx: PromptContext = data.context.context
    return {
        "id": ctx.prompt_id,
        "version": "1",
        "variables": {"poem_style": ctx.poem_style},
    }

agent = Agent(name="Prompted assistant", prompt=build_prompt)
result = await Runner.run(
    agent,
    "Say hello",
    context=PromptContext(prompt_id="pmpt_123", poem_style="limerick")
)
```

Context

Agents are generic on their `context` type. Context is a dependency-injection tool: it's an object you create and pass to `Runner.run()`, that is passed to every agent, tool, handoff etc, and it serves as a grab bag of dependencies and state for the agent run. You can provide any Python object as the context.

Read the [context guide](#) for the full `RunContextWrapper` surface, shared usage tracking, nested `tool_input`, and serialization caveats.

```

from dataclasses import dataclass

@dataclass
class Purchase:
    id: str

@dataclass
class UserContext:
    name: str
    uid: str
    is_pro_user: bool

    async def fetch_purchases(self) -> list[Purchase]:
        # implement your logic here
        return []

agent = Agent[UserContext](
    ...,
)

```

Output types

By default, agents produce plain text (i.e. `str`) outputs. If you want the agent to produce a particular type of output, you can use the `output_type` parameter. A common choice is to use [Pydantic](#) objects, but we support any type that can be wrapped in a Pydantic [TypeAdapter](#) - dataclasses, lists, TypedDict, etc.

```

from pydantic import BaseModel
from agents import Agent

class CalendarEvent(BaseModel):
    name: str
    date: str
    participants: list[str]

agent = Agent(
    name="Calendar extractor",
    instructions="Extract calendar events from text",
)

```

```
    output_type=CalendarEvent,  
)
```

Note

When you pass an `output_type`, that tells the model to use [structured outputs](#) instead of regular plain text responses.

Multi-agent system design patterns

There are many ways to design multi-agent systems, but we commonly see two broadly applicable patterns:

1. **Manager (agents as tools):** A central manager/orchestrator invokes specialized sub-agents as tools and retains control of the conversation.
2. **Handoffs:** Peer agents hand off control to a specialized agent that takes over the conversation. This is decentralized.

See [our practical guide to building agents](#) for more details.

Manager (agents as tools)

The `customer_facing_agent` handles all user interaction and invokes specialized sub-agents exposed as tools. Read more in the [tools](#) documentation.

```
from agents import Agent  
  
booking_agent = Agent(...)  
refund_agent = Agent(...)  
  
customer_facing_agent = Agent(  
    name="Customer-facing agent",  
    instructions=(  
        "Handle all direct user communication. "  
        "Call the relevant tools when specialized expertise is needed  
    ),
```

```
tools=[
    booking_agent.as_tool(
        tool_name="booking_expert",
        tool_description="Handles booking questions and requests."
    ),
    refund_agent.as_tool(
        tool_name="refund_expert",
        tool_description="Handles refund questions and requests."
    )
],
)
```

Handoffs

Configured handoff targets are sub-agents to which the agent can delegate. When a handoff occurs, the delegated agent receives the conversation history and takes over the conversation. This pattern enables modular, specialized agents that excel at a single task. Read more in the [handoffs](#) documentation.

```
from agents import Agent

booking_agent = Agent(...)
refund_agent = Agent(...)

triage_agent = Agent(
    name="Triage agent",
    instructions=(
        "Help the user with their questions. "
        "If they ask about booking, hand off to the booking agent. "
        "If they ask about refunds, hand off to the refund agent."
    ),
    handoffs=[booking_agent, refund_agent],
)
```

Dynamic instructions

In most cases, you can provide instructions when you create the agent. However, you can also provide dynamic instructions via a function. The function will receive

the agent and context, and must return the prompt. Both regular and `async` functions are accepted.

```
from agents import Agent, RunContextWrapper

def dynamic_instructions(
    context: RunContextWrapper[UserContext], agent: Agent[UserContext]
) -> str:
    return f"The user's name is {context.context.name}. Help them wit

agent = Agent[UserContext](
    name="Triage agent",
    instructions=dynamic_instructions,
)
```

Lifecycle events (hooks)

Sometimes, you want to observe the lifecycle of an agent. For example, you may want to log events, pre-fetch data, or record usage when certain events occur.

There are two hook scopes:

- `RunHooks` observe the entire `Runner.run(...)` invocation, including handoffs to other agents.
- `AgentHooks` are attached to a specific agent instance via `agent.hooks`.

The callback context also changes depending on the event:

- Agent start/end hooks receive `AgentHookContext`, which wraps your original context and carries the shared run usage state.
- LLM, tool, and handoff hooks receive `RunContextWrapper`.

Typical hook timing:

- `on_agent_start`: when a specific agent begins running; `on_agent_end`: when that agent finishes producing a final output.

- `on_llm_start` / `on_llm_end`: immediately around each model call.
- `on_tool_start` / `on_tool_end`: around each local tool invocation. For function tools, the hook `context` is typically a `ToolContext`, so you can inspect tool-call metadata such as `tool_call_id`.
- `on_handoff`: when control moves from one agent to another.

Use `RunHooks` when you want a single observer for the whole workflow, and `AgentHooks` when you want lifecycle callbacks scoped to a specific agent.

```
from agents import Agent, RunHooks, Runner

class LoggingHooks(RunHooks):
    async def on_agent_start(self, context, agent):
        print(f"Starting {agent.name}")

    async def on_llm_end(self, context, agent, response):
        print(f"{agent.name} produced {len(response.output)} output i

    async def on_agent_end(self, context, agent, output):
        print(f"{agent.name} finished with usage: {context.usage}")

agent = Agent(name="Assistant", instructions="Be concise.")
result = await Runner.run(agent, "Explain quines", hooks=LoggingHooks)
print(result.final_output)
```

For the full callback surface, see the [Lifecycle API reference](#).

Guardrails

Guardrails allow you to run checks/validations on user input in parallel to the agent running, and on the agent's output once it is produced. For example, you could screen the user's input and agent's output for relevance. Read more in the [guardrails](#) documentation.

Cloning/copying agents

By using the `clone()` method on an agent, you can duplicate an Agent, and optionally change any properties you like.

```
pirate_agent = Agent(
    name="Pirate",
    instructions="Write like a pirate",
    model="gpt-5.6-sol",
)

robot_agent = pirate_agent.clone(
    name="Robot",
    instructions="Write like a robot",
)
```

`clone()` uses `dataclasses.replace`, so it performs a shallow copy. A list attribute that you do not override, such as `tools`, `handoffs`, `mcp_servers`, `input_guardrails`, or `output_guardrails`, remains the exact list held by the original agent. Mutating that list through either agent therefore affects both agents. To give the clone an independent list container, pass a new list, for example `pirate_agent.clone(tools=[*pirate_agent.tools, extra_tool])`. The entries copied into that new list remain the same tool or handoff objects unless you replace those entries too.

Forcing tool use

Supplying a list of tools doesn't always mean the LLM will use a tool. You can force tool use by setting `ModelSettings.tool_choice`. Valid values are:

1. `auto`, which allows the LLM to decide whether or not to use a tool.
2. `required`, which requires the LLM to use a tool (but it can intelligently decide which tool).
3. `none`, which requires the LLM to *not* use a tool.

4. Setting a specific string e.g. `my_tool`, which requires the LLM to use that specific tool.

When you are using OpenAI Responses tool search, named tool choices are more limited: you cannot target bare namespace names or deferred-only tools with `tool_choice`, and `tool_choice="tool_search"` does not target `ToolSearchTool`. In those cases, prefer `auto` or `required`. See [Hosted tool search](#) for the Responses-specific constraints.

```
from agents import Agent, ModelSettings
from agents.decorators import tool

@tool
def get_weather(city: str) -> str:
    """Returns weather info for the specified city."""
    return f"The weather in {city} is sunny"

agent = Agent(
    name="Weather Agent",
    instructions="Retrieve weather details.",
    tools=[get_weather],
    model_settings=ModelSettings(tool_choice="get_weather")
)
```

Tool use behavior

The `tool_use_behavior` parameter in the `Agent` configuration controls how tool outputs are handled:

- `"run_llm_again"`: The default. Tools are run, and the LLM processes the results to produce a final response.
- `"stop_on_first_tool"`: The output of the first tool call is used as the final response, without further LLM processing.

```
from agents import Agent
from agents.decorators import tool

@tool
```

```
def get_weather(city: str) -> str:
    """Returns weather info for the specified city."""
    return f"The weather in {city} is sunny"

agent = Agent(
    name="Weather Agent",
    instructions="Retrieve weather details.",
    tools=[get_weather],
    tool_use_behavior="stop_on_first_tool"
)
```

- `StopAtTools(stop_at_tool_names=[...])` : Stops if any specified tool is called, using its output as the final response.

```
from agents import Agent
from agents.agent import StopAtTools
from agents.decorators import tool

@tool
def get_weather(city: str) -> str:
    """Returns weather info for the specified city."""
    return f"The weather in {city} is sunny"

@tool
def sum_numbers(a: int, b: int) -> int:
    """Adds two numbers."""
    return a + b

agent = Agent(
    name="Stop At Stock Agent",
    instructions="Get weather or sum numbers.",
    tools=[get_weather, sum_numbers],
    tool_use_behavior=StopAtTools(stop_at_tool_names=["get_weather"])
)
```

- `ToolsToFinalOutputFunction` : A custom function that processes tool results and decides whether to end the run with a final output or continue processing with the LLM.

```
from agents import Agent, FunctionToolResult, RunContextWrapper
from agents.agent import ToolsToFinalOutputResult
```

```

from agents.decorators import tool
from typing import List, Any

@tool
def get_weather(city: str) -> str:
    """Returns weather info for the specified city."""
    return f"The weather in {city} is sunny"

def custom_tool_handler(
    context: RunContextWrapper[Any],
    tool_results: List[FunctionToolResult]
) -> ToolsToFinalOutputResult:
    """Processes tool results to decide final output."""
    for result in tool_results:
        if result.output and "sunny" in result.output:
            return ToolsToFinalOutputResult(
                is_final_output=True,
                final_output=f"Final weather: {result.output}"
            )
    return ToolsToFinalOutputResult(
        is_final_output=False,
        final_output=None
    )

agent = Agent(
    name="Weather Agent",
    instructions="Retrieve weather details.",
    tools=[get_weather],
    tool_use_behavior=custom_tool_handler
)

```

Note

To prevent infinite loops, the framework automatically resets `tool_choice` to "auto" after a tool call. This behavior is configurable via `agent.reset_tool_choice`. The infinite loop is because tool results are sent to the LLM, which then generates another tool call because of `tool_choice`, ad infinitum.

Handoffs

Handoffs allow an agent to delegate tasks to another agent. This is particularly useful in scenarios where different agents specialize in distinct areas. For example, a customer support app might have agents that each specifically handle tasks like order status, refunds, FAQs, etc.

Handoffs are represented as tools to the LLM. So if there's a handoff to an agent named `Refund Agent`, the tool would be named `transfer_to_refund_agent`.

Creating a handoff

All agents have a `handoffs` param, which can either take an `Agent` directly, or a `Handoff` object that customizes the Handoff.

If you pass plain `Agent` instances, their `handoff_description` (when set) is appended to the default tool description. Use it to hint when the model should pick that handoff without writing a full `handoff()` object.

You can create a handoff using the `handoff()` function provided by the Agents SDK. This function allows you to specify the agent to hand off to, along with optional overrides and input filters.

Basic usage

Here's how you can create a simple handoff:

```
from agents import Agent, handoff

billing_agent = Agent(name="Billing agent")
refund_agent = Agent(name="Refund agent")
```

```
1
trriage_agent = Agent(name="Triage agent", handoffs=[billing_agent, ha
```

- 1 You can use the agent directly (as in `billing_agent`), or you can use the `handoff()` function.

Customizing handoffs via the `handoff()` function

The `handoff()` function lets you customize things.

- `agent` : This is the agent to which things will be handed off.
- `tool_name_override` : By default, the `Handoff.default_tool_name()` function is used, which resolves to `transfer_to_<agent_name>`. You can override this.
- `tool_description_override` : Override the default tool description from `Handoff.default_tool_description()`
- `on_handoff` : A callback function executed when the handoff is invoked. This is useful for things like kicking off some data fetching as soon as you know a handoff is being invoked. This function receives the agent context, and can optionally also receive LLM generated input. The input data is controlled by the `input_type` param.
- `input_type` : The schema for the handoff tool-call arguments. When set, the parsed payload is passed to `on_handoff`.
- `input_filter` : This lets you filter the input received by the next agent. See below for more.
- `is_enabled` : Whether the handoff is enabled. This can be a boolean or a function that returns a boolean, allowing you to dynamically enable or disable the handoff at runtime.
- `nest_handoff_history` : Optional per-handoff override for the RunConfig-level `nest_handoff_history` setting. If `None`, the value defined in the active run configuration is used instead.

The `handoff()` helper always transfers control to the specific `agent` you passed in. If you have multiple possible destinations, register one handoff per destination

and let the model choose among them. Use a custom `Handoff` only when your own handoff code must decide which agent to return at invocation time.

```
from agents import Agent, handoff, RunContextWrapper

def on_handoff(ctx: RunContextWrapper[None]):
    print("Handoff called")

agent = Agent(name="My agent")

handoff_obj = handoff(
    agent=agent,
    on_handoff=on_handoff,
    tool_name_override="custom_handoff_tool",
    tool_description_override="Custom description",
)
```

Handoff inputs

In certain situations, you want the LLM to provide some data when it calls a handoff. For example, imagine a handoff to an "Escalation agent". You might want the model to provide a reason so you can log it.

```
from pydantic import BaseModel

from agents import Agent, handoff, RunContextWrapper

class EscalationData(BaseModel):
    reason: str

async def on_handoff(ctx: RunContextWrapper[None], input_data: EscalationData):
    print(f"Escalation agent called with reason: {input_data.reason}")

agent = Agent(name="Escalation agent")

handoff_obj = handoff(
    agent=agent,
    on_handoff=on_handoff,
```

```
    input_type=EscalationData,
)
```

`input_type` describes the arguments for the handoff tool call itself. The SDK exposes that schema to the model as the handoff tool's `parameters`, validates the returned JSON locally, and passes the parsed value to `on_handoff`.

`is_enabled` is evaluated while the SDK prepares the available handoffs, before the model returns handoff arguments, so it cannot authorize values inside an argument-bearing handoff. When authorization depends on the parsed fields, perform the check at the start of `on_handoff`, before any application side effects. If authorization fails, raise instead of returning; the SDK continues the transfer after `on_handoff` returns successfully. Tool input guardrails apply to function tools, not handoffs.

It does not replace the next agent's main input, and it does not choose a different destination. The `handoff()` helper still transfers to the specific agent you wrapped, and the receiving agent still sees the conversation history unless you change it with an `input_filter` or nested handoff history settings.

`input_type` is also separate from `RunContextWrapper.context`. Use `input_type` for metadata the model decides at handoff time, not for application state or dependencies you already have locally.

When to use `input_type`

Use `input_type` when the handoff needs a small piece of model-generated metadata such as `reason`, `language`, `priority`, or `summary`. For example, a triage agent can hand off to a refund agent with `{ "reason": "duplicate_charge", "priority": "high" }`, and `on_handoff` can log or persist that metadata before the refund agent takes over.

Choose a different mechanism when the goal is different:

- Put existing application state and dependencies in `RunContextWrapper.context`. See the [context guide](#).

- Use `input_filter`, `RunConfig.nest_handoff_history`, or `RunConfig.handoff_history_mapper` if you want to change what history the receiving agent sees.
- Register one handoff per destination if there are multiple possible specialists. `input_type` can add metadata to the chosen handoff, but it does not dispatch between destinations.
- If you want structured input for a nested specialist without transferring the conversation, prefer `Agent.as_tool(parameters=...)`. See [tools](#).

Input filters

When a handoff occurs, it's as though the new agent takes over the conversation, and gets to see the entire previous conversation history. If you want to change this, you can set an `input_filter`. An input filter is a function that receives the existing input via a `HandoffInputData`, and must return a new `HandoffInputData`.

`HandoffInputData` includes:

- `input_history`: the input history before `Runner.run(...)` started.
- `pre_handoff_items`: items generated before the agent turn where the handoff was invoked.
- `new_items`: items generated during the current turn, including the handoff call and handoff output items.
- `input_items`: optional items to forward to the next agent instead of `new_items`, allowing you to filter model input while keeping `new_items` intact for session history.
- `run_context`: the active `RunContextWrapper` at the time the handoff was invoked.

Nested handoff history is available as an opt-in beta and is disabled by default while we stabilize it. When you enable `RunConfig.nest_handoff_history`, the runner compacts summarizable history into ordered assistant summary

segments while preserving lossless message items in their original positions. Each generated summary segment uses the `<CONVERSATION HISTORY>` wrapper, and later handoffs flatten earlier generated segments before rebuilding the ordered transcript. Sessions, `RunState`, and `RunResult.to_input_list()` track exact message occurrences moved into this SDK-default history so those occurrences are not appended twice; separate identical messages are still preserved. You can provide your own mapping function via `RunConfig.handoff_history_mapper` to return the exact list of input items for the next agent instead of using the built-in segmentation. The opt-in applies only when neither the handoff's `input_filter` nor the active run's `RunConfig.handoff_input_filter` is set, so existing code that already customizes the payload (including the examples in this repository) keeps its current behavior without changes. You can override the nesting behaviour for a single handoff by passing `nest_handoff_history=True` or `False` to `handoff(...)`, which sets `Handoff.nest_handoff_history`. If you just need to change the wrapper text for generated summary segments, call `set_conversation_history_wrappers` before running your agents. Call `reset_conversation_history_wrappers` before a later run when you need to restore the default wrappers.

If both the handoff and the active `RunConfig.handoff_input_filter` define a filter, the per-handoff `input_filter` takes precedence for that specific handoff.

Note

Handoffs stay within a single run. Input guardrails still apply only to the first agent in the chain, and output guardrails only to the agent that produces the final output. Use tool guardrails when you need checks around each custom function-tool call inside the workflow.

There are some common patterns (for example removing all tool calls from the history), which are implemented for you in `agents.extensions.handoff_filters`

```
from agents import Agent, handoff
from agents.extensions import handoff_filters
```

```
agent = Agent(name="FAQ agent")

handoff_obj = handoff(
    agent=agent,
    input_filter=handoff_filters.remove_all_tools, 1
)
```

- 1 This will automatically remove all tool-related items from the history when `FAQ agent` is called.

Recommended prompts

To make sure that LLMs understand handoffs properly, we recommend including information about handoffs in your agents. We have a suggested prefix in `agents.extensions.handoff_prompt.RECOMMENDED_PROMPT_PREFIX`, or you can call `agents.extensions.handoff_prompt.prompt_with_handoff_instructions` to automatically add recommended data to your prompts.

```
from agents import Agent
from agents.extensions.handoff_prompt import RECOMMENDED_PROMPT_PREFIX

billing_agent = Agent(
    name="Billing agent",
    instructions=f"""{RECOMMENDED_PROMPT_PREFIX}
    <Fill in the rest of your prompt here>.""",
)
```

Guardrails

Guardrails enable you to do checks and validations of user input and agent output. For example, imagine you have an agent that uses a very smart (and hence slow/expensive) model to help with customer requests. You wouldn't want malicious users to ask the model to help them with their math homework. So, you can run a guardrail with a fast/cheap model. If the guardrail detects malicious usage, it can immediately raise an error, saving time and money. Blocking execution guarantees that the expensive model does not start; with parallel execution, the expensive model may already have started before the guardrail completes. See "Execution modes" below for details.

There are two kinds of guardrails:

1. Input guardrails run on the initial user input
2. Output guardrails run on the final agent output

Workflow boundaries

Guardrails are attached to agents and tools, but they do not all run at the same points in a workflow:

- **Input guardrails** run only for the first agent in the chain.
- **Output guardrails** run only for the agent that produces the final output.
- **Tool guardrails** run on every guarded function-tool invocation, including local MCP tools when their server configures guardrails, with input guardrails before execution and output guardrails after execution.

If you need checks before and/or after each custom function-tool call in a workflow that includes managers, handoffs, or delegated specialists, use tool guardrails instead of relying only on agent-level input/output guardrails.

Input guardrails

Input guardrails run in 3 steps:

1. First, the guardrail receives the same input passed to the agent.
2. Next, the guardrail function runs to produce a `GuardrailFunctionOutput`, which is then wrapped in an `InputGuardrailResult`
3. Finally, we check if `.tripwire_triggered` is true. If true, an `InputGuardrailTripwireTriggered` exception is raised, so you can appropriately respond to the user or handle the exception.

Note

Input guardrails are intended to run on user input, so an agent's guardrails only run if the agent is the *first* agent. You might wonder, why is the `guardrails` property on the agent instead of passed to `Runner.run`? It's because guardrails tend to be related to the actual Agent - you'd run different guardrails for different agents, so colocating the code is useful for readability.

Execution modes

Input guardrails support two execution modes:

- **Parallel execution** (default, `run_in_parallel=True`): The guardrail runs concurrently with the agent's execution. This provides the best latency since both start at the same time. However, if the guardrail's tripwire is triggered, the agent may have already consumed tokens and executed tools before being cancelled.
- **Blocking execution** (`run_in_parallel=False`): The guardrail runs and completes *before* the agent starts. If the guardrail tripwire is triggered, the agent never executes, preventing token consumption and tool execution. This is ideal for cost optimization and when you want to avoid potential side effects from tool calls.

Output guardrails

Output guardrails run in 3 steps:

1. First, the guardrail receives the output produced by the agent.
2. Next, the guardrail function runs to produce a `GuardrailFunctionOutput`, which is then wrapped in an `OutputGuardrailResult`
3. Finally, we check if `.tripwire_triggered` is true. If true, an `OutputGuardrailTripwireTriggered` exception is raised, so you can appropriately respond to the user or handle the exception.

Note

Output guardrails are intended to run on the final agent output, so an agent's guardrails only run if the agent is the *last* agent. Similar to the input guardrails, we do this because guardrails tend to be related to the actual Agent - you'd run different guardrails for different agents, so colocating the code is useful for readability.

Output guardrails always run after the agent completes, so they don't support the `run_in_parallel` parameter.

An output tripwire and an exception raised by the guardrail function have different session behavior. A tripwire rejects the candidate final output. When a tripwire fires, the runner asks the configured session to persist already-completed tool call and tool output items, together with any reasoning context required to replay those calls, while excluding the rejected candidate final output. The runner applies this tripwire rule to both streaming and non-streaming runs. When the guardrail function raises an exception instead of returning a tripwire result, the runner treats the verdict as unknown and asks the configured session to persist the completed final-turn items before surfacing the guardrail exception. If that session write also fails, the session write error takes precedence. Streaming runs use the same persistence ordering as non-streaming runs and raise the terminal

exception from `stream_events()`. An immediate `RunResultStreaming.cancel()` call while the output guardrail is running cancels the in-flight guardrail and does not start a final-turn session write.

Terminal function-tool output needs additional handling because the tool has already run before the agent-level output guardrail checks the value. When `Agent.tool_use_behavior` makes that tool result the final output and an output tripwire rejects it, the SDK retains a replay-valid function call/output pair only when it can rebuild the pair from validated fields. The retained `function_call_output` payload is replaced with the default text "Output withheld by an output guardrail."; the original tool-output payload is not retained in the session, `RunState`, streamed result state, or sandbox memory input. The SDK does retain validated function-call metadata required for replay, including the function arguments, so that metadata can contain data that also appeared in the rejected output. Current-response `OutputGuardrailResult` objects also replace `agent_output` with the resolved placeholder and clear `output_info`. Current-response `ToolOutputGuardrailResult` objects preserve the allow/reject behavior type but replace payload-bearing `output_info` and rejection messages with the same placeholder. Earlier accepted turns and guardrail results remain unchanged. If the response contains reasoning or another shape that the SDK cannot sanitize safely, the SDK discards the complete current-response suffix instead of retaining the rejected output payload. A guardrail function that raises an exception has not returned a rejection verdict, so the completed terminal-tool turn follows the exception persistence behavior described above.

Set `RunConfig.output_guardrail_blocked_message` to a non-empty string or a synchronous formatter when your application needs a different data-free placeholder. The formatter receives `OutputGuardrailBlockedMessageArgs` with the SDK default, the guardrail name, the agent, and the active run context. It never receives the rejected tool output or guardrail `output_info`. The returned text is persisted and replayed wherever the SDK retains the sanitized terminal-tool turn, so keep it free of sensitive data and do not copy secrets from the run context. If the formatter raises, returns `None`, returns an empty or non-string

value, or produces an awaitable, the SDK uses the default placeholder. Async formatter functions are rejected when `RunConfig` is constructed.

```
from agents import OutputGuardrailBlockedMessageArgs, RunConfig

def blocked_message(args: OutputGuardrailBlockedMessageArgs[dict[str, str]],
                    return f"Output blocked by policy: {args.guardrail_name}."

run_config = RunConfig(output_guardrail_blocked_message=blocked_messa
```

Tool guardrails

Tool guardrails wrap `FunctionTool` instances and let you validate or block calls to those tools before and after execution. They are configured on the tool itself and run every time that tool is invoked.

- Input tool guardrails run before the tool executes and can skip the call, replace the output with a message, or raise a tripwire.
- Output tool guardrails run after the tool executes and can replace the output or raise a tripwire.
- If a function tool requires approval, input tool guardrails normally run after approval and immediately before execution. Set `RunConfig.tool_execution` to `ToolExecutionConfig(pre_approval_tool_input_guardrails=True)` when you want those input checks to run before the pending approval interruption is emitted. Calls that pass this pre-approval check are still checked again after approval before the tool executes.
- Tool guardrails use the `FunctionTool` execution pipeline. You can attach them directly to a custom tool created with `tool` or `function_tool`. You can also set `tool_input_guardrails` and `tool_output_guardrails` on a local MCP server; the SDK attaches those lists to every tool exposed by that server. Handoffs run through the SDK's handoff pipeline rather than the function-tool pipeline, so tool guardrails do not apply to the handoff call itself. Hosted tools

(`WebSearchTool`, `FileSearchTool`, `HostedMCPTool`, `CodeInterpreterTool`, `ImageGenerationTool`) and built-in execution tools (`ComputerTool`, `ShellTool`, `ApplyPatchTool`, `LocalShellTool`) do not use this guardrail pipeline, and `Agent.as_tool()` does not currently expose tool-guardrail options directly. See [MCP server tool guardrails](#) for the local MCP configuration.

See the code snippet below for details.

Tripwires

If an agent input or output fails a guardrail, the guardrail can signal this with a tripwire. The runner immediately raises an `InputGuardrailTripwireTriggered` or `OutputGuardrailTripwireTriggered` exception and halts agent execution. Tool guardrails use the corresponding `ToolInputGuardrailTripwireTriggered` and `ToolOutputGuardrailTripwireTriggered` exceptions.

For agent-level tripwires, the exception's `guardrail_result` identifies the guardrail that triggered the tripwire. For an input tripwire raised by the runner, `exception.run_data.input_guardrail_results` contains every input guardrail result completed before the run stopped, including the result that triggered the tripwire. Output tripwires provide the equivalent accumulated results through `exception.run_data.output_guardrail_results`.

Tool tripwire exceptions instead expose the triggering `guardrail` and `output` directly. Their `run_data.tool_input_guardrail_results` and `run_data.tool_output_guardrail_results` lists preserve results accumulated from completed turns before the failure; the triggering result is available through the exception's `output`. Other runner-managed failures, such as `MaxTurnsExceeded`, also preserve completed tool guardrail results in these lists. After `stream_events()` raises an exception, the streamed result exposes the same accumulated agent and tool guardrail result lists. `run_data` can be `None` when an exception is raised outside a runner-managed execution path.

Implementing a guardrail

You need to provide a function that receives input, and returns a `GuardrailFunctionOutput`. In this example, we'll do this by running an Agent under the hood.

```
from pydantic import BaseModel
from agents import (
    Agent,
    GuardrailFunctionOutput,
    InputGuardrailTripwireTriggered,
    RunContextWrapper,
    Runner,
    TResponseInputItem,
)
from agents.decorators import input_guardrail

class MathHomeworkOutput(BaseModel):
    is_math_homework: bool
    reasoning: str

guardrail_agent = Agent( ❶
    name="Guardrail check",
    instructions="Check if the user is asking you to do their math ho
    output_type=MathHomeworkOutput,
)

@input_guardrail
async def math_guardrail( ❷
    ctx: RunContextWrapper[None], agent: Agent, input: str | list[TRe
) -> GuardrailFunctionOutput:
    result = await Runner.run(guardrail_agent, input, context=ctx.con

    return GuardrailFunctionOutput(
        output_info=result.final_output, ❸
        tripwire_triggered=result.final_output.is_math_homework,
    )

agent = Agent( ❹
    name="Customer support agent",
```

```

        instructions="You are a customer support agent. You help customer
        input_guardrails=[math_guardrail],
    )

    async def main():
        # This should trip the guardrail
        try:
            await Runner.run(agent, "Hello, can you help me solve for x:
            print("Guardrail didn't trip - this is unexpected")

        except InputGuardrailTripwireTriggered:
            print("Math homework guardrail tripped")

```

- 1 We'll use this agent in our guardrail function.
- 2 This is the guardrail function that receives the agent's input/context, and returns the result.
- 3 We can include extra information in the guardrail result.
- 4 This is the actual agent that defines the workflow.

Output guardrails are similar.

```

from pydantic import BaseModel
from agents import (
    Agent,
    GuardrailFunctionOutput,
    OutputGuardrailTripwireTriggered,
    RunContextWrapper,
    Runner,
)
from agents.decorators import output_guardrail
class MessageOutput(BaseModel): 1
    response: str

class MathOutput(BaseModel): 2
    reasoning: str
    is_math: bool

guardrail_agent = Agent(
    name="Guardrail check",
    instructions="Check if the output includes any math.",

```

```

        output_type=MathOutput,
    )

    @output_guardrail
    async def math_guardrail( ③
        ctx: RunContextWrapper, agent: Agent, output: MessageOutput
    ) -> GuardrailFunctionOutput:
        result = await Runner.run(guardrail_agent, output.response, conte

        return GuardrailFunctionOutput(
            output_info=result.final_output,
            tripwire_triggered=result.final_output.is_math,
        )

    agent = Agent( ④
        name="Customer support agent",
        instructions="You are a customer support agent. You help customer
        output_guardrails=[math_guardrail],
        output_type=MessageOutput,
    )

    async def main():
        # This should trip the guardrail
        try:
            await Runner.run(agent, "Hello, can you help me solve for x:
            print("Guardrail didn't trip - this is unexpected")

        except OutputGuardrailTripwireTriggered:
            print("Math output guardrail tripped")

```

- ① This is the actual agent's output type.
- ② This is the guardrail's output type.
- ③ This is the guardrail function that receives the agent's output, and returns the result.
- ④ This is the actual agent that defines the workflow.

Lastly, here are examples of tool guardrails.

```

import json
from agents import (

```

```
    Agent,
    Runner,
    ToolGuardrailFunctionOutput,
)
from agents.decorators import tool, tool_input_guardrail, tool_output

@tool_input_guardrail
def block_secrets(data):
    args = json.loads(data.context.tool_arguments or "{}")
    if "sk-" in json.dumps(args):
        return ToolGuardrailFunctionOutput.reject_content(
            "Remove secrets before calling this tool."
        )
    return ToolGuardrailFunctionOutput.allow()

@tool_output_guardrail
def redact_output(data):
    text = str(data.output or "")
    if "sk-" in text:
        return ToolGuardrailFunctionOutput.reject_content("Output con
    return ToolGuardrailFunctionOutput.allow()

@tool(
    tool_input_guardrails=[block_secrets],
    tool_output_guardrails=[redact_output],
)
def classify_text(text: str) -> str:
    """Classify text for internal routing."""
    return f"length:{len(text)}"

agent = Agent(name="Classifier", tools=[classify_text])
result = Runner.run_sync(agent, "hello world")
print(result.final_output)
```

Sessions

The Agents SDK provides built-in session memory to automatically maintain conversation history across multiple agent runs, eliminating the need to manually handle `.to_input_list()` between turns.

Sessions stores conversation history for a specific session, allowing agents to maintain context without requiring explicit manual memory management. This is particularly useful for building chat applications or multi-turn conversations where you want the agent to remember previous interactions.

Use sessions when you want the SDK to manage client-side memory for you. In the same run, a session cannot be combined with the run-level continuation options `conversation_id`, `previous_response_id`, or `auto_previous_response_id`. If you want OpenAI server-managed continuation instead, choose one of those mechanisms rather than layering a session on top.

Quick start

```
from agents import Agent, Runner, SQLiteSession

# Create agent
agent = Agent(
    name="Assistant",
    instructions="Reply very concisely.",
)

# Create a session instance with a session ID
session = SQLiteSession("conversation_123")

# First turn
result = await Runner.run(
    agent,
    "What city is the Golden Gate Bridge in?",
    session=session
```

```
)
print(result.final_output) # "San Francisco"

# Second turn - agent automatically remembers previous context
result = await Runner.run(
    agent,
    "What state is it in?",
    session=session
)
print(result.final_output) # "California"

# Also works with synchronous runner
result = Runner.run_sync(
    agent,
    "What's the population?",
    session=session
)
print(result.final_output) # "Approximately 39 million"
```

Resuming interrupted runs with the same session

If a run pauses for approval, resume it with the same session instance (or another instance configured with the same session ID and the same underlying storage backend) so the resumed turn continues the same stored conversation history.

```
result = await Runner.run(agent, "Delete temporary files that are no

if result.interruptions:
    state = result.to_state()
    for interruption in result.interruptions:
        state.approve(interruption)
    result = await Runner.run(agent, state, session=session)
```

Core session behavior

When session memory is enabled:

1. **Before each run:** The runner automatically retrieves the conversation history for the session and prepends it to the input items.

2. **After each run:** All new items generated during the run (user input, assistant responses, tool calls, etc.) are automatically stored in the session.
3. **Context preservation:** Each subsequent run with the same session includes the full conversation history, allowing the agent to maintain context.

This eliminates the need to manually call `.to_input_list()` and manage conversation state between runs.

Control how history and new input merge

When you pass a session, the runner normally prepares model input as:

1. Session history (retrieved from `session.get_items(...)`)
2. New turn input

Use `RunConfig.session_input_callback` to customize that merge step before the model call. The callback receives two lists:

- `history`: The retrieved session history (already normalized into input-item format)
- `new_input`: The current turn's new input items

Return the final list of input items that should be sent to the model.

The callback receives copies of both lists, so you can safely mutate them. The returned list controls the model input for that turn, but the SDK still persists only items that belong to the new turn. Reordering or filtering old history therefore does not cause old session items to be saved again as fresh input.

```
from agents import Agent, RunConfig, Runner, SQLiteSession

def keep_recent_history(history, new_input):
    # Keep only the last 10 history items, then append the new turn.
    return history[-10:] + new_input
```

```
agent = Agent(name="Assistant")
session = SQLiteSession("conversation_123")

result = await Runner.run(
    agent,
    "Continue from the latest updates only.",
    session=session,
    run_config=RunConfig(session_input_callback=keep_recent_history),
)
```

Use this when you need custom pruning, reordering, or selective inclusion of history without changing how the session stores items. If you need a later final pass immediately before the model call, use `call_model_input_filter` from the [running agents guide](#).

Limiting retrieved history

Use `SessionSettings` to control how much history is fetched before each run.

- `SessionSettings(limit=None)` (default): retrieve all available session items
- `SessionSettings(limit=N)`: retrieve only the most recent `N` items

You can apply this per run via `RunConfig.session_settings`:

```
from agents import Agent, RunConfig, Runner, SessionSettings, SQLiteS

agent = Agent(name="Assistant")
session = SQLiteSession("conversation_123")

result = await Runner.run(
    agent,
    "Summarize our recent discussion.",
    session=session,
    run_config=RunConfig(session_settings=SessionSettings(limit=50)),
)
```

If your session implementation exposes default session settings, each non-`None` value in `RunConfig.session_settings` overrides the corresponding default for

that run. This is useful for long conversations where you want to cap retrieval size without changing the session's default behavior.

Memory operations

Basic operations

Sessions supports several operations for managing conversation history:

```
from agents import SQLiteSession

session = SQLiteSession("user_123", "conversations.db")

# Get all items in a session
items = await session.get_items()

# Add new items to a session
new_items = [
    {"role": "user", "content": "Hello"},
    {"role": "assistant", "content": "Hi there!"}
]
await session.add_items(new_items)

# Remove and return the most recent item
last_item = await session.pop_item()
print(last_item) # {"role": "assistant", "content": "Hi there!"}

# Clear all items from a session
await session.clear_session()
```

Using pop_item for corrections

The `pop_item` method is particularly useful when you want to undo or modify the last item in a conversation:

```
from agents import Agent, Runner, SQLiteSession

agent = Agent(name="Assistant")
session = SQLiteSession("correction_example")
```

```
# Initial conversation
result = await Runner.run(
    agent,
    "What's 2 + 2?",
    session=session
)
print(f"Agent: {result.final_output}")

# User wants to correct their question
assistant_item = await session.pop_item() # Remove agent's response
user_item = await session.pop_item() # Remove user's question

# Ask a corrected question
result = await Runner.run(
    agent,
    "What's 2 + 3?",
    session=session
)
print(f"Agent: {result.final_output}")
```

Built-in session implementations

The SDK provides several session implementations for different use cases:

Choose a built-in session implementation

Use this table to pick a starting point before reading the detailed examples below.

Session type	Best for	Notes
<code>SQLiteSession</code>	Local development and simple apps	Built-in, lightweight, file-backed or in-memory
<code>AsyncSQLiteSession</code>	Async SQLite with <code>aiosqlite</code>	Extension backend with async driver support
<code>RedisSession</code>	Shared memory across workers/services	Good for low-latency distributed deployments

Session type	Best for	Notes
<code>SQLAlchemySession</code>	Production apps with existing databases	Works with SQLAlchemy-supported databases
<code>MongoDBSession</code>	Apps already using MongoDB or needing multi-process storage	Async pymongo; atomic sequence counter for ordering
<code>DaprSession</code>	Cloud-native deployments with Dapr sidecars	Supports multiple state stores plus TTL and consistency controls
<code>OpenAIConversationsSession</code>	Server-managed storage in OpenAI	OpenAI Conversations API-backed history
<code>OpenAIResponsesCompactionSession</code>	Long conversations with automatic compaction	Wrapper around another session backend
<code>AdvancedSQLiteSession</code>	SQLite plus branching/analytics	Heavier feature set; see dedicated page
<code>EncryptedSession</code>	Encryption + TTL on top of another session	Wrapper; choose an underlying backend first

Some implementations have dedicated pages with additional details; those are linked inline in their subsections.

If you are implementing a Python server for ChatKit, use a `chatkit.store.Store` implementation for ChatKit's thread and item persistence. Agents SDK sessions such as `SQLAlchemySession` manage SDK-side conversation history, but they are not a drop-in replacement for ChatKit's store. See the [chatkit-python guide on implementing your ChatKit data store](#).

OpenAI Conversations API sessions

Use [OpenAI's Conversations API](#) through `OpenAIConversationsSession`.

```
from agents import Agent, Runner, OpenAIConversationsSession

# Create agent
agent = Agent(
    name="Assistant",
    instructions="Reply very concisely.",
)

# Create a new conversation
session = OpenAIConversationsSession()

# Optionally resume a previous conversation by passing a conversation
# session = OpenAIConversationsSession(conversation_id="conv_123")

# Start conversation
result = await Runner.run(
    agent,
    "What city is the Golden Gate Bridge in?",
    session=session
)
print(result.final_output) # "San Francisco"

# Continue the conversation
result = await Runner.run(
    agent,
    "What state is it in?",
    session=session
)
print(result.final_output) # "California"
```

OpenAI Responses compaction sessions

Use `OpenAIResponsesCompactionSession` to compact stored conversation history with the Responses API (`responses.compact`). It wraps an underlying session and can automatically compact after each turn based on `should_trigger_compaction` . Do not wrap `OpenAIConversationsSession` with it; those two features manage history in different ways.

Typical usage (auto-compaction)

```
from agents import Agent, Runner, SQLiteSession
from agents.memory import OpenAIResponsesCompactionSession

underlying = SQLiteSession("conversation_123")
session = OpenAIResponsesCompactionSession(
    session_id="conversation_123",
    underlying_session=underlying,
)

agent = Agent(name="Assistant")
result = await Runner.run(agent, "Hello", session=session)
print(result.final_output)
```

By default, after each turn, the SDK checks whether the compaction candidate meets the threshold and compacts only if it does.

When automatic compaction runs, the SDK waits for it before `Runner.run(...)` returns or the streamed event iterator closes. Usage reported by the compaction request contributes to that run's `Usage` totals. By default, a manual `run_compaction()` call made later has no enclosing run context and does not update the completed run's usage object.

`compaction_mode="previous_response_id"` uses Responses API response IDs retained by the compaction session and works best while that response chain remains available. `compaction_mode="input"` rebuilds the compaction request from the current session items instead, which is useful when the response chain is unavailable or you want the session contents to be the source of truth. The default `"auto"` chooses the safest available option.

If your agent runs with `ModelSettings(store=False)`, the Responses API does not retain the last response for later lookup. In that stateless setup, the default `"auto"` mode falls back to input-based compaction instead of relying on `previous_response_id`. See [examples/memory/compaction_session_stateless_example.py](https://openai.github.io/openai-agents-python/sessions/examples/memory/compaction_session_stateless_example.py) for a complete example.

auto-compaction can block streaming

Compaction clears and rewrites the session history, so the SDK waits for compaction to finish before considering the run complete. In streaming mode, this means `run.stream_events()` can stay open for a few seconds after the last output token if compaction is heavy.

`OpenAIResponsesCompactionSession.run_compaction()` treats the clear-and-rewrite operation as a recoverable replacement at the wrapper boundary. If replacement fails or is cancelled after the underlying history changes, the wrapper attempts to restore the previous history and waits for that recovery attempt to settle before the original exception or cancellation reaches the caller. If the underlying backend also fails during recovery, the previous history can remain unrestored and the SDK logs the recovery failure. The wrapper serializes `add_items()`, `pop_item()`, and `clear_session()` with the entire compaction operation, including the remote request and the replacement or recovery phase. A concurrent wrapper mutation waits for compaction and then applies to the resulting history instead of being overwritten. Automatic compaction also records which wrapper generation belongs to the run; if another run changes the history before that compaction starts, the SDK skips the stale compaction instead of replacing newer history. Do not mutate the underlying session directly while compaction is running because direct mutations bypass these wrapper guarantees.

If you want low-latency streaming or fast turn-taking, disable auto-compaction and call `run_compaction()` yourself between turns (or during idle time). You can decide when to force compaction based on your own criteria.

```
from agents import Agent, Runner, SQLiteSession
from agents.memory import OpenAIResponsesCompactionSession

underlying = SQLiteSession("conversation_123")
session = OpenAIResponsesCompactionSession(
    session_id="conversation_123",
    underlying_session=underlying,
    # Disable triggering the auto compaction
    should_trigger_compaction=lambda _: False,
)
```

```
agent = Agent(name="Assistant")
result = await Runner.run(agent, "Hello", session=session)

# Decide when to compact (e.g., on idle, every N turns, or size thres
await session.run_compaction({"force": True})
```

SQLite sessions

The default, lightweight session implementation using SQLite:

```
from agents import SQLiteSession

# In-memory database (lost when process ends)
session = SQLiteSession("user_123")

# Persistent file-based database
session = SQLiteSession("user_123", "conversations.db")

# Use the session
result = await Runner.run(
    agent,
    "Hello",
    session=session
)
```

Async SQLite sessions

Use `AsyncSQLiteSession` when you want SQLite persistence backed by `aiosqlite`.

```
pip install aiosqlite
```

```
from agents import Agent, Runner
from agents.extensions.memory import AsyncSQLiteSession

agent = Agent(name="Assistant")
session = AsyncSQLiteSession("user_123", db_path="conversations.db")
result = await Runner.run(agent, "Hello", session=session)
```

Redis sessions

Use `RedisSession` for shared session memory across multiple workers or services.

```
pip install openai-agents[redis]
```

```
from agents import Agent, Runner
from agents.extensions.memory import RedisSession

agent = Agent(name="Assistant")
session = RedisSession.from_url(
    "user_123",
    url="redis://localhost:6379/0",
)
result = await Runner.run(agent, "Hello", session=session)
await session.close()
```

`from_url(...)` creates and owns the Redis client. After `close()`, the session is terminal and subsequent session operations raise `RuntimeError`; repeated or concurrent `close()` calls are safe. If your application already manages a Redis client, construct `RedisSession(...)` directly with `redis_client=...`. In that case, `close()` is a no-op and the caller retains both client ownership and session usability.

SQLAlchemy sessions

Production-ready Agents SDK session persistence using any SQLAlchemy-supported database:

```
from agents.extensions.memory import SQLAlchemySession

# Using database URL
session = SQLAlchemySession.from_url(
    "user_123",
    url="postgresql+asyncpg://user:pass@localhost/db",
    create_tables=True
)
```

```
# Using existing engine
from sqlalchemy.ext.asyncio import create_async_engine
engine = create_async_engine("postgresql+asyncpg://user:pass@localhost")
session = SQLAlchemySession("user_123", engine=engine, create_tables=
```

See [SQLAlchemy Sessions](#) for detailed documentation.

Dapr sessions

Use `DaprSession` when you already run Dapr sidecars or want to switch the configured state-store backend without changing your agent code.

```
pip install openai-agents[dapr]
```

```
from agents import Agent, Runner
from agents.extensions.memory import DaprSession

agent = Agent(name="Assistant")

async with DaprSession.from_address(
    "user_123",
    state_store_name="statestore",
    dapr_address="localhost:50001",
) as session:
    result = await Runner.run(agent, "Hello", session=session)
    print(result.final_output)
```

Notes:

- `from_address(...)` creates and owns the Dapr client for you. If your app already manages one, construct `DaprSession(...)` directly with `dapr_client=...`.
- Exiting the context or calling `close()` makes an owned-client session terminal; subsequent session operations raise `RuntimeError`, while repeated or concurrent `close()` calls are safe. With an injected client, `close()` is a no-op and the session remains usable.

- If the backing state store supports TTL, pass `ttl=...` so it automatically applies TTL expiration to the session data.
- Pass `consistency=DAPR_CONSISTENCY_STRONG` to request strong consistency on state writes and deletes (store-dependent). Note that wire-level read consistency requires upstream Dapr Python client support for setting consistency on `get_state`.
- The Dapr Python SDK also checks the HTTP sidecar endpoint. In local development, start Dapr with `--dapr-http-port 3500` as well as the gRPC port used in `dapr_address`.
- See [examples/memory/dapr_session_example.py](#) for a full setup walkthrough, including local components and troubleshooting.

MongoDB sessions

Use `MongoDBSession` for applications that already use MongoDB or need horizontally-scalable, multi-process session storage.

```
pip install openai-agents[mongodb]
```

```
from agents import Agent, Runner
from agents.extensions.memory import MongoDBSession

agent = Agent(name="Assistant")

# Create from URI - owns the client and closes it when session.close()
session = MongoDBSession.from_uri(
    "user-123",
    uri="mongodb://localhost:27017",
    database="agents",
)
result = await Runner.run(agent, "Hello", session=session)
print(result.final_output)
await session.close()
```

Notes:

- `from_uri(...)` creates and owns the `AsyncMongoClient` and closes it on `session.close()`. An owned-client session is terminal after `close()`, and subsequent session operations raise `RuntimeError`. If your application already manages a client, construct `MongoDBSession(...)` directly with `client=...`; in that case, `session.close()` is a no-op, the caller retains responsibility for the client lifecycle, and the session remains usable.
- Connect to [MongoDB Atlas](#) by passing an `mongodb+srv://user:password@cluster.example.mongodb.net` URI to `from_uri(...)` with no other changes.
- Two collections are used and both names are configurable via `sessions_collection=` (default `agent_sessions`) and `messages_collection=` (default `agent_messages`). Indexes are created automatically on first use. Each non-empty `add_items()` call writes one logical-batch document whose monotonically increasing `seq` orders the batch by its final item; legacy per-item message documents remain readable. A logical batch must fit within MongoDB's single-document size limit; an oversized batch fails atomically without storing a partial batch.
- Use `await session.ping()` to verify connectivity before your first run.

Advanced SQLite sessions

Enhanced SQLite sessions with conversation branching, usage analytics, and structured queries:

```
from agents.extensions.memory import AdvancedSQLiteSession

# Create with advanced features
session = AdvancedSQLiteSession(
    session_id="user_123",
    db_path="conversations.db",
    create_tables=True
)

# Automatic usage tracking
result = await Runner.run(agent, "Hello", session=session)
await session.store_run_usage(result) # Track token usage
```

```
# Conversation branching
await session.create_branch_from_turn(2) # Branch from turn 2
```

See [Advanced SQLite Sessions](#) for detailed documentation.

Encrypted sessions

Transparent encryption wrapper for any session implementation:

```
from agents.extensions.memory import EncryptedSession, SQLAlchemySession

# Create underlying session
underlying_session = SQLAlchemySession.from_url(
    "user_123",
    url="sqlite+aiosqlite:///conversations.db",
    create_tables=True
)

# Wrap with encryption and TTL
session = EncryptedSession(
    session_id="user_123",
    underlying_session=underlying_session,
    encryption_key="your-secret-key",
    ttl=600 # 10 minutes
)

result = await Runner.run(agent, "Hello", session=session)
```

See [Encrypted Sessions](#) for detailed documentation.

Other session types

There are a few more built-in options. Please refer to `examples/memory/` and source code under `extensions/memory/`.

Operational patterns

Session ID naming

Use meaningful session IDs that help you organize conversations:

- User-based: `"user_12345"`
- Thread-based: `"thread_abc123"`
- Context-based: `"support_ticket_456"`

Memory persistence

- Use in-memory SQLite (`SQLiteSession("session_id")`) for temporary conversations
- Use file-based SQLite (`SQLiteSession("session_id", "path/to/db.sqlite")`) for persistent conversations
- Use async SQLite (`AsyncSQLiteSession("session_id", db_path="...")`) when you need an `aiosqlite`-based implementation
- Use Redis-backed sessions (`RedisSession.from_url("session_id", url="redis://...")`) for shared, low-latency session memory
- Use SQLAlchemy-powered sessions (`SQLAlchemySession("session_id", engine=engine, create_tables=True)`) for production systems with existing databases supported by SQLAlchemy
- Use MongoDB sessions (`MongoDBSession.from_uri("session_id", uri="mongodb://localhost:27017")`) for applications already using MongoDB or needing multi-process, horizontally-scalable session storage
- Use Dapr state store sessions (`DaprSession.from_address("session_id", state_store_name="statestore", dapr_address="localhost:50001")`) for production cloud-native deployments with built-in telemetry, tracing, and data isolation and support for 30+ database backends
- Use OpenAI-hosted storage (`OpenAIConversationsSession()`) when you prefer to store history in the OpenAI Conversations API
- Use encrypted sessions (`EncryptedSession(session_id, underlying_session, encryption_key)`) to wrap any session with transparent encryption and TTL-based expiration

- Consider implementing custom session backends for other production systems (for example, Django) for more advanced use cases

Multiple sessions

```
from agents import Agent, Runner, SQLiteSession

agent = Agent(name="Assistant")

# Different sessions maintain separate conversation histories
session_1 = SQLiteSession("user_123", "conversations.db")
session_2 = SQLiteSession("user_456", "conversations.db")

result1 = await Runner.run(
    agent,
    "Help me with my account",
    session=session_1
)
result2 = await Runner.run(
    agent,
    "What are my charges?",
    session=session_2
)
```

Session sharing

```
# Different agents can share the same session
support_agent = Agent(name="Support")
billing_agent = Agent(name="Billing")
session = SQLiteSession("user_123")

# Both agents will see the same conversation history
result1 = await Runner.run(
    support_agent,
    "Help me with my account",
    session=session
)
result2 = await Runner.run(
    billing_agent,
    "What are my charges?",
    session=session
)
```

```
        session=session  
    )
```

Complete example

Here's a complete example showing session memory in action:

```
import asyncio  
from agents import Agent, Runner, SQLiteSession  
  
async def main():  
    # Create an agent  
    agent = Agent(  
        name="Assistant",  
        instructions="Reply very concisely.",  
    )  
  
    # Create a session instance that will persist across runs  
    session = SQLiteSession("conversation_123", "conversation_history")  
  
    print("=== Sessions Example ===")  
    print("The agent will remember previous messages automatically.\n")  
  
    # First turn  
    print("First turn:")  
    print("User: What city is the Golden Gate Bridge in?")  
    result = await Runner.run(  
        agent,  
        "What city is the Golden Gate Bridge in?",  
        session=session  
    )  
    print(f"Assistant: {result.final_output}")  
    print()  
  
    # Second turn - the agent will remember the previous conversation  
    print("Second turn:")  
    print("User: What state is it in?")  
    result = await Runner.run(  
        agent,  
        "What state is it in?",  
        session=session  
    )
```

```

    )
    print(f"Assistant: {result.final_output}")
    print()

    # Third turn - continuing the conversation
    print("Third turn:")
    print("User: What's the population of that state?")
    result = await Runner.run(
        agent,
        "What's the population of that state?",
        session=session
    )
    print(f"Assistant: {result.final_output}")
    print()

    print("=== Conversation Complete ===")
    print("Notice how the agent remembered the context from previous")
    print("Sessions automatically handles conversation history.")

if __name__ == "__main__":
    asyncio.run(main())

```

Custom session implementations

You can implement your own session memory by creating a class that structurally follows the `Session` protocol. You do not need to inherit from `SessionABC`; define `session_id` and `session_settings`, and implement the four history methods directly:

```

from agents import Agent, Runner, SessionSettings
from agents.items import TResponseInputItem

class MyCustomSession:
    """Custom session implementation following the Session protocol."""

    session_settings: SessionSettings | None = None

    def __init__(self, session_id: str) -> None:
        self.session_id = session_id

```

```

        self.items: list[TResponseInputItem] = []

    async def get_items(self, limit: int | None = None) -> list[TResp
        if limit is None:
            return list(self.items)
        if limit <= 0:
            return []
        return list(self.items[-limit:])

    async def add_items(self, items: list[TResponseInputItem]) -> Non
        self.items.extend(items)

    async def pop_item(self) -> TResponseInputItem | None:
        return self.items.pop() if self.items else None

    async def clear_session(self) -> None:
        self.items.clear()

# Use your custom session
agent = Agent(name="Assistant")
result = await Runner.run(
    agent,
    "Hello",
    session=MyCustomSession("my_session")
)

```

Accessing run context from a custom session

The Agents SDK can pass the active `RunContextWrapper` to a custom session for tenant routing, authorization, or other app-specific storage decisions. For the Agents SDK to pass the wrapper, add an explicitly named, keyword-compatible `wrapper` parameter to all four history methods:

```

from typing import Any

from agents import RunContextWrapper
from agents.items import TResponseInputItem

class ContextAwareSession:

```

```
async def get_items(
    self,
    limit: int | None = None,
    *,
    wrapper: RunContextWrapper[Any] | None = None,
) -> list[TResponseInputItem]: ...

async def add_items(
    self,
    items: list[TResponseInputItem],
    *,
    wrapper: RunContextWrapper[Any] | None = None,
) -> None: ...

async def pop_item(
    self,
    *,
    wrapper: RunContextWrapper[Any] | None = None,
) -> TResponseInputItem | None: ...

async def clear_session(
    self,
    *,
    wrapper: RunContextWrapper[Any] | None = None,
) -> None: ...
```

The Agents SDK enables this integration only when `get_items`, `add_items`, `pop_item`, and `clear_session` all declare `wrapper`. A generic `**kwargs` parameter does not satisfy this signature check. Existing session implementations that omit `wrapper` keep their released call shape and continue to work without changes.

Community session implementations

The community has developed additional session implementations:

Package	Description
openai-django-sessions	Django ORM-based sessions for any Django-supported database (PostgreSQL, MySQL, SQLite, and more)

If you've built a session implementation, please feel free to submit a documentation PR to add it here!

API reference

For detailed API documentation, see:

- [Session](#) - Protocol interface
- [OpenAIConversationsSession](#) - OpenAI Conversations API implementation
- [OpenAIResponsesCompactionSession](#) - Responses API compaction wrapper
- [SQLiteSession](#) - Basic SQLite implementation
- [AsyncSQLiteSession](#) - Async SQLite implementation based on `aiosqlite`
- [RedisSession](#) - Redis-backed session implementation
- [SQLAlchemySession](#) - SQLAlchemy-powered implementation
- [MongoDBSession](#) - MongoDB-backed session implementation
- [DaprSession](#) - Dapr state store implementation
- [AdvancedSQLiteSession](#) - Enhanced SQLite with branching and analytics
- [EncryptedSession](#) - Encrypted wrapper for any session

Tracing

The Agents SDK includes built-in tracing, collecting a comprehensive record of events during an agent run: LLM generations, tool calls, handoffs, guardrails, and even custom events that occur. Using the [Traces dashboard](#), you can debug, visualize, and monitor your workflows during development and in production.

Note

Tracing is enabled by default. You can disable it in three common ways:

1. You can globally disable tracing by setting the env var

```
OPENAI_AGENTS_DISABLE_TRACING=1
```

2. You can globally disable tracing in code with

```
set_tracing_disabled(True)
```

3. You can disable tracing for a single run by setting

```
agents.run.RunConfig.tracing_disabled to True
```

Tracing is unavailable for organizations that use OpenAI's APIs under a Zero Data Retention (ZDR) policy.

Traces and spans

- **Traces** represent a single end-to-end operation of a "workflow". They're composed of Spans. Traces have the following properties:
 - `workflow_name`: This is the name of the logical workflow or app. For example "Code generation" or "Customer service".
 - `trace_id`: A unique ID for the trace. Automatically generated if you don't pass one. Must have the format `trace_<32_alphanumeric>`.

- `group_id` : Optional group ID, to link multiple traces from the same conversation. For example, you might use a chat thread ID.
- `disabled` : If True, the trace will not be recorded.
- `metadata` : Optional metadata for the trace.
- **Spans** represent operations that have a start and end time. Spans have:
 - `started_at` and `ended_at` timestamps.
 - `trace_id` , to represent the trace they belong to
 - `parent_id` , which points to the parent Span of this Span (if any)
 - `span_data` , which is information about the Span. For example, `AgentSpanData` contains information about the Agent, `GenerationSpanData` contains information about the LLM generation, etc.

Default tracing

By default, the SDK traces the following:

- The entire `Runner.{run, run_sync, run_streamed}()` is wrapped in a `trace()` .
- Each runner invocation is wrapped in a `task_span()` .
- Each model turn is wrapped in a `turn_span()` .
- Each time an agent runs, it is wrapped in `agent_span()`
- LLM generations are wrapped in `generation_span()`
- Function tool calls are each wrapped in `function_span()`
- Guardrails are wrapped in `guardrail_span()`
- Handoffs are wrapped in `handoff_span()`
- Audio inputs (speech-to-text) are wrapped in a `transcription_span()`
- Audio outputs (text-to-speech) are wrapped in a `speech_span()`

- The SDK may parent related audio spans under a `speech_group_span()`

By default, the trace name is the literal string `Agent workflow`. You can set this name if you use `trace`, or you can configure the name and other properties with the `RunConfig`.

If you want a more compact hierarchy, disable the automatic task and turn spans for a run. Agent, generation, function, guardrail, handoff, and custom spans are still recorded.

```
from agents import RunConfig, Runner

result = await Runner.run(
    agent,
    "Hello",
    run_config=RunConfig(tracing={"include_task_and_turn_spans": False})
)
```

In addition, you can set up [custom trace processors](#) to push traces to other destinations (as a replacement, or secondary destination).

Long-running workers and immediate exports

The default `BatchTraceProcessor` exports traces in the background every few seconds, or sooner when the in-memory queue reaches its size trigger, and also performs a final flush when the process exits. In long-running workers such as Celery, RQ, Dramatiq, or FastAPI background tasks, this means traces are usually exported automatically without any extra code, but they may not appear in the Traces dashboard immediately after each job finishes.

If you need an immediate delivery guarantee at the end of a unit of work, call `flush_traces()` after the trace context exits.

```
from agents import Runner, flush_traces, trace

@celery_app.task
```

```
def run_agent_task(prompt: str):
    try:
        with trace("celery_task"):
            result = Runner.run_sync(agent, prompt)
        return result.final_output
    finally:
        flush_traces()
```

```
from fastapi import BackgroundTasks, FastAPI
from agents import Runner, flush_traces, trace

app = FastAPI()

def process_in_background(prompt: str) -> None:
    try:
        with trace("background_job"):
            Runner.run_sync(agent, prompt)
    finally:
        flush_traces()

@app.post("/run")
async def run(prompt: str, background_tasks: BackgroundTasks):
    background_tasks.add_task(process_in_background, prompt)
    return {"status": "queued"}
```

`flush_traces()` blocks until currently buffered traces and spans are exported, so call it after `trace()` closes to avoid flushing a partially built trace. You can skip this call when the default export latency is acceptable.

Disabling tracing prevents the default provider from creating new traces and spans, but it does not discard data that its processors already buffered.

`flush_traces()` continues to flush that buffered data after tracing has been disabled through `set_tracing_disabled(True)` or `OPENAI_AGENTS_DISABLE_TRACING=1`.

Higher level traces

Sometimes, you might want multiple calls to `run()` to be part of a single trace. You can do this by wrapping the entire code in a `trace()`.

```
from agents import Agent, Runner, trace

async def main():
    agent = Agent(name="Joke generator", instructions="Tell funny jokes")

    with trace("Joke workflow"): ❶
        first_result = await Runner.run(agent, "Tell me a joke")
        second_result = await Runner.run(agent, f"Rate this joke: {first_result}")
        print(f"Joke: {first_result.final_output}")
        print(f"Rating: {second_result.final_output}")
```

- ❶ Because the two calls to `Runner.run` are wrapped in a `with trace()`, both runs become part of one overall trace instead of each creating a separate trace.

Creating traces

You can use the `trace()` function to create a trace. Traces need to be started and finished. You have two options to do so:

1. **Recommended:** use the trace as a context manager, i.e. `with trace(...) as my_trace`. This will automatically start and end the trace at the right time.
2. You can also manually call `trace.start()` and `trace.finish()`.

The current trace is tracked via a Python `contextvar`. This means that it works with concurrency automatically. If you manually start and finish a trace, pass `mark_as_current` to `start()` and `reset_current` to `finish()` to update the current trace.

Creating spans

You can use the various `*_span()` methods to create a span. In general, you don't need to manually create spans. A `custom_span()` function is available for tracking

custom span information.

Spans are automatically part of the current trace, and are nested under the nearest current span, which is tracked via a Python `contextvar`.

Sensitive data

Certain spans may capture potentially sensitive data.

The `generation_span()` stores the inputs/outputs of the LLM generation, and `function_span()` stores the inputs/outputs of function calls. These may contain sensitive data, so you can disable capturing that data via `RunConfig.trace_include_sensitive_data`.

For an approval-gated function tool, a span that pauses for approval does not store the SDK's internal result wrapper as tool output. If the application rejects the call with a custom rejection message, the function span stores that message as output and error text only when `trace_include_sensitive_data` is `True`. When the setting is `False`, the span omits the output and uses the generic error text `Tool execution rejected`.

Similarly, Audio spans include base64-encoded PCM data for input and output audio by default. You can disable capturing this audio data by configuring `VoicePipelineConfig.trace_include_sensitive_audio_data`.

By default, `trace_include_sensitive_data` is `True`. You can set the default without code by exporting the `OPENAI_AGENTS_TRACE_INCLUDE_SENSITIVE_DATA` environment variable to `true/1` or `false/0` before running your app.

When `trace_include_sensitive_data` is `False`, Responses model spans omit the request input and response output. For calls to an official OpenAI endpoint, the spans still include the Responses API `response_id` as correlation metadata. The SDK omits that identifier from redacted spans for custom endpoints.

Custom tracing processors

The high level architecture for tracing is:

- At initialization, we create a global `TraceProvider`, which is responsible for creating traces.
- We configure the `TraceProvider` with a `BatchTraceProcessor` that sends traces/spans in batches to a `BackendSpanExporter`, which exports the spans and traces to the OpenAI backend in batches.

To customize this default setup, to send traces to alternative or additional backends or modifying exporter behavior, you have two options:

1. `add_trace_processor()` lets you add an **additional** trace processor that will receive traces and spans as they are ready. This lets you do your own processing in addition to sending traces to OpenAI's backend.
2. `set_trace_processors()` lets you **replace** the default processors with your own trace processors. This means traces will not be sent to the OpenAI backend unless you include a `TracingProcessor` that does so.

Tracing with non-OpenAI models

When using non-OpenAI models, you can provide an OpenAI API key to the tracing exporter to enable free tracing in the OpenAI Traces dashboard without disabling tracing. See the [Third-party adapters](#) section in the Models guide for adapter selection and setup caveats.

```
import os
from agents import set_tracing_export_api_key, Agent
from agents.extensions.models.any_llm_model import AnyLLMModel

tracing_api_key = os.environ["OPENAI_API_KEY"]
set_tracing_export_api_key(tracing_api_key)

model = AnyLLMModel(
    model="your-provider/your-model-name",
    api_key="your-api-key",
)
```

```
agent = Agent(  
    name="Assistant",  
    model=model,  
)
```

If you only need a different tracing key for a single run, pass it via `RunConfig` instead of changing the global exporter.

```
from agents import Runner, RunConfig  
  
await Runner.run(  
    agent,  
    input="Hello",  
    run_config=RunConfig(tracing={"api_key": "sk-tracing-123"}),  
)
```

Additional notes

- View free traces at OpenAI Traces dashboard.

Ecosystem integrations

The following community and vendor integrations support the tracing API surface of the OpenAI Agents SDK.

External tracing processors list

- [Weights & Biases](#)
- [Arize Phoenix](#)
- [Future AGI](#)
- [MLflow \(self-hosted/OSS\)](#)
- [MLflow \(Databricks hosted\)](#)
- [Braintrust](#)
- [Pydantic Logfire](#)

- [AgentOps](#)
- [Scorecard](#)
- [Respan](#)
- [LangSmith](#)
- [Maxim AI](#)
- [Comet Opik](#)
- [Langfuse](#)
- [Langtrace](#)
- [Okahu-Monocle](#)
- [Galileo](#)
- [Portkey AI](#)
- [LangDB AI](#)
- [Agenta](#)
- [PostHog](#)
- [Traccia](#)
- [PromptLayer](#)
- [HoneyHive](#)
- [Asqav](#)
- [Datadog](#)
- [Latitude](#)
- [DProvenanceKit](#)
- [Tuning Engines](#)